

Code Explanation

Databricks Delta Live Tables (DLT) Pipeline Code Explanation

This code is a Databricks Delta Live Tables (DLT) pipeline written in Python. It is designed to read customer and order data from CSV files, clean and transform the data, and then create several tables to store the processed data. The pipeline implements Slowly Changing Dimension (SCD) Type 2 logic to track changes in customer information over time.

Overview of the Code

The code defines a DLT pipeline that consists of several tables: `customer\_bronze`, `order\_bronze`, `customer\_silver`, `order\_silver`, `ordersummary`, and `customeraggregatespend`. The pipeline reads data from CSV files, cleans and transforms the data, and then creates the tables to store the processed data.

Step 1: Define Schemas for Customer and Order Data

This step defines the schemas for the customer and order data using PySpark's `StructType` and `StructField` classes. The schemas specify the structure of the data, including the column names, data types, and whether each column is nullable.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])
```

Step 2: Read Source CSV Data from Volume

This step reads the customer and order data from CSV files stored in a Databricks volume. The `@dlt.table` decorator is used to define the tables `customer\_bronze` and `order\_bronze`, which store the raw data from the CSV files.

```
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data from CSV files"
)
def customer_bronze():
    return (
        spark.read
        .option("header", "true")
        .schema(customer_schema)
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customer
data")
    )
```

```

@dlt.table(
    name="order_bronze",
    comment="Raw order data from CSV files"
)
def order_bronze():
    return (
        spark.read
        .option("header", "true")
        .schema(order_schema)
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdat
a")
    )

```

Step 3: Clean Customer Data

This step cleans the customer data by removing null values and duplicates. The `@dlt.table` decorator is used to define the `customer\_silver` table, which stores the cleaned customer data.

```

@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data with nulls and duplicates removed"
)
def customer_silver():
    return (
        dlt.read("customer_bronze")
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
        .dropDuplicates(["CustId"])
    )

```

Step 4: Clean Order Data and Calculate Total Amount

This step cleans the order data by removing null values and duplicates, and calculates the total amount for each order. The `@dlt.table` decorator is used to define the `order\_silver` table, which stores the cleaned order data.

```

@dlt.table(
    name="order_silver",
    comment="Cleaned order data with TotalAmount calculated"
)
def order_silver():
    return (
        dlt.read("order_bronze")
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &

```

```

        (F.col("CustId").isNotNull()))
    )
    .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
    .dropDuplicates(["OrderId"])
)

```

Step 5: Create SCD Type 2 Table for Order Summary

This step creates an SCD Type 2 table `ordersummary` that stores the joined customer and order data. The table tracks changes in customer information over time.

```

@dlt.table(
    name="ordersummary",
    table_properties={
        "quality": "silver",
        "delta.enableChangeDataFeed": "true"
    },
    comment="SCD Type 2 table joining customer and order data",
    temporary=False,
    spark_conf={"spark.databricks.delta.schema.autoMerge.enabled": "true"}
)
def ordersummary():
    # Get current data
    current_data = (
        dlt.read("customer_silver")
        .join(
            dlt.read("order_silver"),
            "CustId",
            "inner"
        )
        .select(
            "CustId", "Name", "EmailId", "Region", "OrderId",
            "ItemName", "PricePerUnit", "Qty", "Date", "TotalAmount"
        )
    )

    # Check if the target table exists
    try:
        existing_data = spark.table("gen_ai_poc_databrickscoe.sdsc_wizard.ordersummary")

        # Implement SCD Type 2 logic
        # ...

    except:
        # If table doesn't exist, create initial version with all records as active
        return (
            current_data
            .withColumn("IsActive", F.lit(True))
            .withColumn("StartDate", F.current_date())
            .withColumn("EndDate", F.lit(None).cast("date"))
        )

```

Step 6: Implement SCD Type 2 Logic

This step implements the SCD Type 2 logic to track changes in customer information over time. It checks for changed records, expires old records, creates new active records, and unions all the record sets.

```
# Find records that need to be updated (changed customer info)
changed_records = (
    current_data
    .join(
        existing_data.filter(F.col("IsActive") == True),
        ["CustId", "OrderId"],
        "inner"
    )
    .filter(
        (current_data["Name"] != existing_data["Name"]) |
        (current_data["EmailId"] != existing_data["EmailId"]) |
        (current_data["Region"] != existing_data["Region"])
    )
    .select(existing_data["CustId"], existing_data["OrderId"])
)

# Expire the old records
expired_records = (
    existing_data
    .join(changed_records, ["CustId", "OrderId"], "inner")
    .withColumn("IsActive", F.lit(False))
    .withColumn("EndDate", F.current_date())
)

# Create new active records
new_active_records = (
    current_data
    .join(changed_records, ["CustId", "OrderId"], "inner")
    .withColumn("IsActive", F.lit(True))
    .withColumn("StartDate", F.current_date())
    .withColumn("EndDate", F.lit(None).cast("date"))
)

# Get unchanged records
unchanged_records = (
    existing_data
    .join(changed_records, ["CustId", "OrderId"], "leftanti")
)

# Get completely new records
new_records = (
    current_data
    .join(
        existing_data.select("CustId", "OrderId").distinct(),
        ["CustId", "OrderId"],
        "leftanti"
    )
    .withColumn("IsActive", F.lit(True))
    .withColumn("StartDate", F.current_date())
    .withColumn("EndDate", F.lit(None).cast("date"))
)
```

```
# Union all the record sets
return (
    unchanged_records
    .unionByName(expired_records)
    .unionByName(new_active_records)
    .unionByName(new_records)
)
```

Step 7: Create Customer Aggregate Spend Table

This step creates a table `customeraggregatespend` that stores the aggregated customer spending by name and date.

```
@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date",
    temporary=False
)
def customeraggregatespend():
    return (
        dlt.read("ordersummary")
        .filter(F.col("IsActive") == True)
        .groupBy("Name", "Date")
        .agg(F.sum("TotalAmount").alias("TotalAmount"))
    )
```